1. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

2. To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

Code:

priyanka@DESKTOP-HOISO7A

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_LEN 100

typedef struct {
    char value[MAX_LEN];
    int type;
} Token;

typedef struct {
    char command[MAX_LEN];
    char args[MAX_TOKENS][MAX_LEN];
    int argc;
} Execution;

int tokenize(char *input, Token tokens[]) {
    int n = 0;
    char *p = input;

    while (*p) {
        while (isspace(*p))
            p++;

        if (*p == '\0')
            break;

        if (*p == '|') {
            strcpy(tokens[n].value, "|");
            tokens[n++].type = 2;
            p++;
        } else {
            int i = 0;

            while (*p && !isspace(*p) && *p != '|') {
                if (i < MAX_LEN - 1)
                    tokens[n].value[i++] = *p;
```

priyanka@DESKTOP-HOISO7A

GNU nano 8.7.1

```
                p++;
            }

            tokens[n].value[i] = '\0';
            tokens[n++].type = 1;
        }

        if (n >= MAX_TOKENS)
            break;
    }

    return n;
}

int parse(Token tokens[], int n, Execution *exec) {
    if (n == 0)
        return 0;

    if (tokens[0].type != 1)
        return -1;

    strcpy(exec->command, tokens[0].value);
    exec->argc = 0;

    for (int i = 1; i < n; i++) {
        if (tokens[i].type == 2) {
            if (i == n - 1 || tokens[i + 1].type == 2)
                return -1;
        } else {
            if (exec->argc < MAX_TOKENS)
                strcpy(exec->args[exec->argc++], tokens[i].value);
        }
    }

    return 1;
}

void print_tree(Execution *exec) {
    printf("\nParse Tree\n");
    printf("Command: %s\n", exec->command);
```

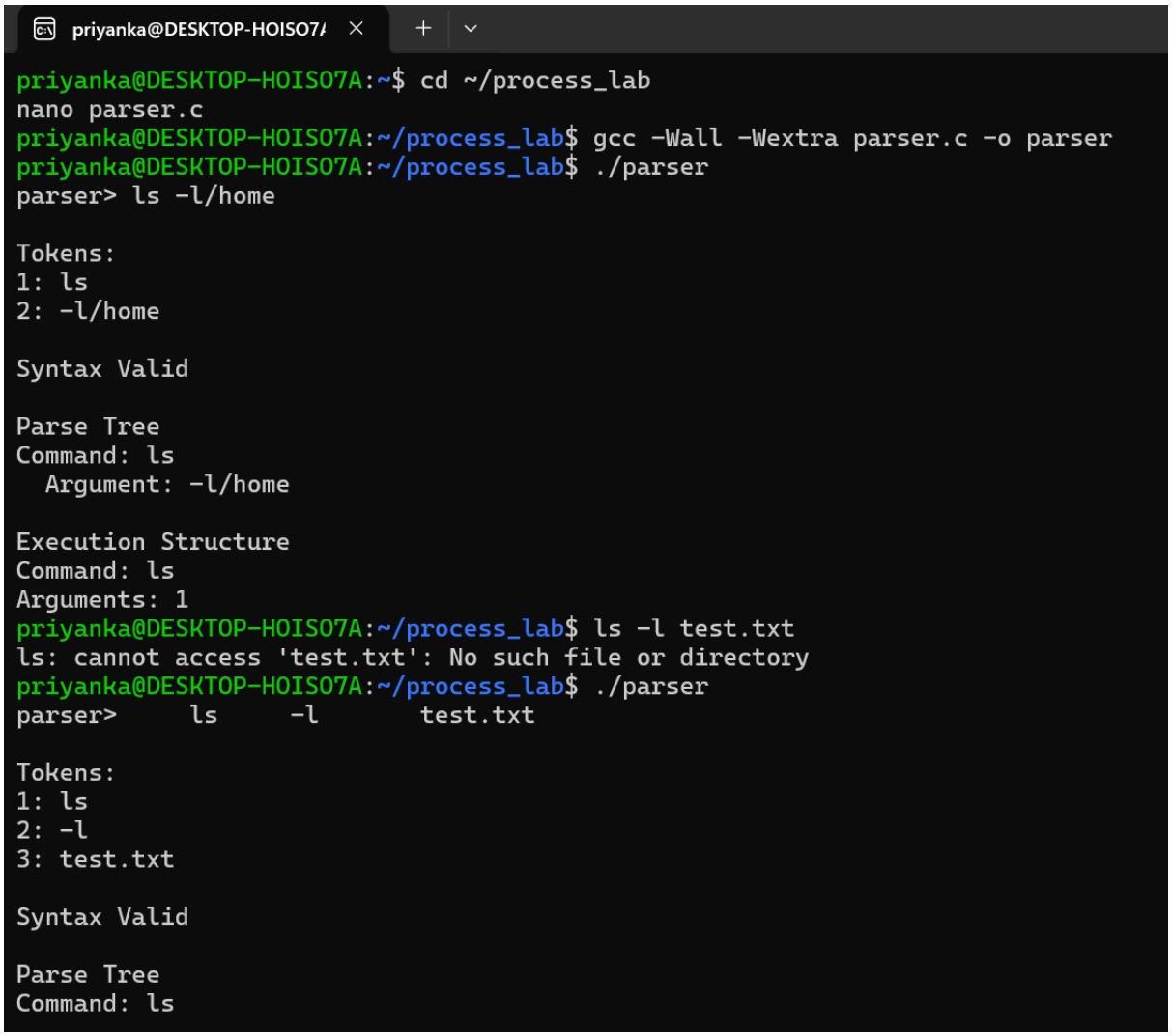
priyanka@DESKTOP-HOISO7A

GNU nano 8.7.1

```
    for (int i = 0; i < exec->argc; i++)
        printf("  Argument: %s\n", exec->args[i]);
}

int main() {
    char input[500];
    Token tokens[MAX_TOKENS];
    Execution exec;

    printf("parser> ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    int count = tokenize(input, tokens);

    if (count == 0) {
        printf("Empty command\n");
        return 0;
    }

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
        printf("%d: %s\n", i + 1, tokens[i].value);

    int result = parse(tokens, count, &exec);

    if (result == -1) {
        printf("\nSyntax Error\n");
        return 1;
    }

    printf("\nSyntax Valid\n");

    print_tree(&exec);

    printf("\nExecution Structure\n");
    printf("Command: %s\n", exec.command);
    printf("Arguments: %d\n", exec.argc);
```

```
    return 0;
}
```

Output:


priyanka@DESKTOP-HOISO7A:~$ cd ~/process_lab
nano parser.c
priyanka@DESKTOP-HOISO7A:~/process_lab$ gcc -Wall -Wextra parser.c -o parser
priyanka@DESKTOP-HOISO7A:~/process_lab$ ./parser
parser> ls -l/home
Tokens:
1: ls
2: -l/home
Syntax Valid
Parse Tree
Command: ls
Argument: -l/home
Execution Structure
Command: ls
Arguments: 1
priyanka@DESKTOP-HOISO7A:~/process_lab$ ls -l test.txt
ls: cannot access 'test.txt': No such file or directory
priyanka@DESKTOP-HOISO7A:~/process_lab$ ./parser
parser> ls -l test.txt
Tokens:
1: ls
2: -l
3: test.txt
Syntax Valid
Parse Tree
Command: ls

```
Command: ls
  Argument: -l
  Argument: test.txt

Execution Structure
Command: ls
Arguments: 2
priyanka@DESKTOP-HOISO7A:~/process_lab$ ./parser
parser> ls -l | grep txt

Tokens:
1: ls
2: -l
3: |
4: grep
5: txt

Syntax Valid

Parse Tree
Command: ls
  Argument: -l
  Argument: grep
  Argument: txt

Execution Structure
Command: ls
Arguments: 3
priyanka@DESKTOP-HOISO7A:~/process_lab$
```